# Problem Set 01

## Objects, References and Basic OOP

Computer Science & Programming (Python)
SUPSI

## Instructions

- Duration: 3 hours (in-class).
- You may use your notes and slides.
- Write clear and readable code.
- When required, answer conceptual questions in comments.

## Exercise 1 — Objects, Identity, and References

Consider the following code:

```
a = [1, 2]
b = a
c = [1, 2]
```

1. Determine whether each statement is `True` or `False`:
   - `a is b`
   - `a == b`
   - `a is c`
   - `a == c`
2. Explain each answer using the concepts of identity and equality.
3. Use `id()` to verify your reasoning.

## Exercise 2 — Mutability and Side Effects

Implement the following functions:

```
def remove_even_inplace(lst):
    """
    Removes all even numbers from lst IN PLACE.
    Returns None.
    """
```

```
def remove_even_new(lst):
    """
    Returns a NEW list without even numbers.
    Does NOT modify the original list.
    """
```

Then answer:

1. Which function produces side effects?
2. How can you verify this using `id()`?
3. Why can modifying a list while iterating over it be dangerous?

## Exercise 3 — Designing a Course Class

Implement a class `Course` with the following requirements.

### Attributes

- `name` (string)
- `ects` (strictly positive integer)
- `grades` (initially empty list)

### Constraints

- `ects` must be strictly positive.
- Grades must be between 1 and 6.
- Invalid values must raise `ValueError`.

### Methods

- `add_grade(value)`
- `average()`
- `is_passed()` (True if average $\geq 4$)
- `__str__()`

In comments, answer:

1. What are the constants of this class?
2. Which attributes are mutable?
3. Why must validation occur inside the class?

## Exercise 4 — Student and Object Composition

Implement a class `Student` with:

### Attributes

- `name`
- `student_id`
- a list of `Course` objects

### Constraints

- A student cannot have two courses with the same name.

### Methods

- `add_course(course)`
- `total_ects_passed()`
- `weighted_average()` (weighted by ECTS)
- `__str__()`

Then:

1. Create at least two students.
2. Add different courses to each.
3. Show that modifying one student does not affect the other.
4. Explain why this works.

## Exercise 5 — The Subtle Bug

Consider the following class:

```
class Student:
    courses = []

    def add_course(self, course):
        self.courses.append(course)
```

1. Create two students.
2. Add a course to only one.
3. What happens?
4. Why does this happen?
5. Rewrite the class to fix the design error.
6. Which object-oriented principle is being violated? Hint: shared state

## Exercise 6 — Advanced Extension (Optional)

Extend the `Student` class:

- Implement `best_course()` returning the course with highest average.
- Implement `__eq__()` for `Course`.

  HELP: *https://medium.com/@DavidHofff/what-is-the-eq-method-in-python-classes-614779526843*

- Make `Student` iterable over its courses.
- Write a main to test the functionalities that you have implemented

Explain briefly how mutability could become dangerous in this design.